MARS KOLONİSİ YAŞAM DESTEK SİMÜLATÖRÜ

Algoritma ve Programlama Dersi – Proje Raporu

Hazırlayan: Mavlyuda Almazova

1. Projenin Amacı

Bu projede, tek karakterli komutlarla kontrol edilen, metin tabanlı bir Mars kolonisi yaşam destek simülatörü C dilinde geliştirilmiştir. Oyuncu, Mars'ta tek başına kalmış bir mühendisi yönetir ve kurtarma ekibi gelene kadar 15 gün hayatta kalmaya çalışır. Programın tamamı **mars_kolonisi.c** dosyasındaki **main** fonksiyonunda, yalnızca temel C yapıları (operatörler, IF–ELSE, SWITCH–CASE, FOR, DO–WHILE) kullanılarak yazılmıştır.

2. Durum Değişkenleri ve Sabitler

Simülasyonun tüm durumu **int** türündeki değişkenlerde tutulur. Maliyetler ve sınırlar **#define** ile sabit olarak tanımlanmıştır; böylece kodun içinde anlamı belirsiz sayılar kullanılmaz.

```
int saglik, enerji, oksijen;          /* 0-100 araliginda       */
int su, yedekParca, modulHasari;      /* hasar 0-100 araliginda */
int kilitMuhurlu = 0;                 /* 0: acik, 1: muhurlu    */
int gun = 1, tur = 0;                 /* gun 1'den baslar       */

#define SON_GUN 15
#define KESIF_ENERJI 15
#define KESIF_OKSIJEN 10
```

Zorluk seviyesine bağlı parametreler (su bulma eşikleri, bulunan su miktarı, yaralanma hasarı, fırtına süresi ve fırtına hasarı) da ayrı değişkenlerde saklanır. Rastgelelik için program başında bir kez **srand(time(NULL))** çağrılır, her olayda **rand() % 100** ile 0–99 arası bir zar üretilir ve zar ile eşik değeri ekrana yazdırılır.

3. Zorluk Seçimi: SWITCH–CASE ve DO–WHILE

Zorluk seviyesi bir **do-while** döngüsü içinde sorulur. Girilen karakter **switch** ile değerlendirilir; **case 'k': case 'K':** şeklindeki fall-through yapısı sayesinde küçük ve büyük harf aynı bloğa düşer. Geçersiz bir giriş **default** bloğunda **gecerli = 0** yapar ve döngü seçimi tekrar ister.

```
do {
    gecerli = 1;
    scanf(" %c", &secim);
    switch (secim) {
        case 'k':
        case 'K':
            saglik = 100; enerji = 100; oksijen = 90; su = 20;
            firtinaTurSayisi = 3; firtinaHasari = 5;  /* ... */
            break;
        /* case 'n'/'N' ve case 'z'/'Z' benzer sekilde */
        default:
            printf("[UYARI] Gecersiz zorluk seviyesi...");
            gecerli = 0;
            break;
    }
} while (gecerli == 0);
```

Zor seviyede başlangıç suyu 8, fırtına 7 tur ve taban hasarı 12 iken; Kolay seviyede su 20, fırtına 3 tur ve taban hasarı 5'tir. Böylece seçilen seviye hem başlangıç değerlerini hem de olay eşiklerini değiştirir.

4. Ana Komut Döngüsü ve Komut Sistemi

Oyunun kendisi bir **do-while** döngüsüdür ve **oyunBitti** değişkeni 0 olduğu sürece devam eder (1: kazandı, 2: kaybetti, 3: Q ile çıktı). Her turda komut okunur ve satırın kalanı **getchar()** ile temizlenir. Komutlar yine fall-through içeren bir **switch** ile yönetilir. **turGecti** bayrağı, komutun tur harcayıp harcamadığını belirtir: yalnızca başarıyla çalışan komutlar bu bayrağı 1 yapar. **D** komutu, tanımsız komutlar (**default**) ve enerji yetersizliği nedeniyle iptal edilen komutlar tur harcamaz.

```
switch (komut) {
    case 'g':
    case 'G':
        if (enerji < KESIF_ENERJI) {
            printf("[IPTAL] Kesif icin en az %d enerji gerekir", ...);
            break;                      /* komut iptal, tur yok */
        }
        ...
        turGecti = 1;
        break;
    ...
    default:
        printf("[UYARI] Tanimsiz komut: '%c'. Tur harcanmadi.\n", komut);
        break;
}
```

5. Komutların Mantığı

G – Keşif: Önce mantıksal VE operatörüyle durum değerlendirilir: **(enerji > 40 && oksijen > 30)** ise yüksek su eşiği, değilse düşük eşik kullanılır. Ardından enerji 15, oksijen 10 azaltılır. Tek bir zar ile üç farklı sonuç IF–ELSE IF–ELSE zinciriyle üretilir: zar su eşiğinin altındaysa su, su eşiği ile **esik + parcaEsik** arasındaysa yedek parça bulunur, aksi halde oyuncu eli boş döner. İkinci bir zar ile VEYA operatörü kullanılarak yaralanma kontrol edilir:

```
zar = rand() % 100;
if (zar < 10 || modulHasari > 70) {
    saglik -= yaralanmaHasari;
}
```

O – Oksijen üretimi: Kullanıcının girdiği su miktarı bir **do-while** ile 2 ile mevcut su arasında olana kadar tekrar sorulur; böylece mevcut sudan fazla miktar girilemez. Tek/çift kontrolü mod operatörüyle yapılır: tek sayıda girilen suyun 1 birimi artık olarak depoda kalır. Her 2 birim su 3 birim oksijene dönüşür; oksijen 100'ü aşarsa fazlası tahliye edilir.

```
artik      = miktar % 2;          /* tekse 1, ciftse 0     */
kullanilan = miktar - artik;
uretilen   = (kullanilan / 2) * 3;
su      -= kullanilan;
oksijen += uretilen;
```

T – Tamir: Yedek parça yoksa tamir iptal edilir. Hasar seviyesi üç kademeye ayrılır ve her kademede harcanan parça ile onarılan hasar farklıdır. Parça yetmezse onarım, eldeki parçayla orantılı olarak **onarim * harcanan / gerekenParca** formülüyle hesaplanır.

```
if (modulHasari <= 30)      { gerekenParca = 1; onarim = 15; }  /* hafif */
else if (modulHasari <= 70) { gerekenParca = 2; onarim = 30; }  /* orta  */
else                        { gerekenParca = 3; onarim = 45; }  /* agir  */
```

D – Durum raporu: Tüm değişkenler yazdırılır. Oksijen < 25, sağlık < 30, hasar > 70 ve enerji < 20 durumları için ayrı ayrı IF blokları uyarı verir. Tüm değerler güvenli aralıktaysa tek bir birleşik koşulla "TUM SISTEMLER NORMAL" yazdırılır:

```
if (oksijen >= 25 && saglik >= 30 && modulHasari <= 70 && enerji >= 20) {
    printf(">>> TUM SISTEMLER NORMAL <<<\n");
}
```

U – Dinlenme: Enerji artışı **20 + (100 - enerji) / 4** formülüyle hesaplanır; yorgun oyuncu daha çok dinlenir. Uyurken 4 oksijen tüketilir. Sağlık için iç içe IF kullanılır: oksijen 20'nin altındaysa sağlık 5 azalır; değilse ve sağlık tam değilse **10 + saglik / 10** kadar artar.

```
if (oksijen < 20) {
    saglik -= 5;
} else {
    if (saglik < MAKS_DEGER) {
        saglik += saglikArtisi;
    }
}
```

6. Kum Fırtınası: FOR Döngüsü, break ve continue

K komutu, zorluğa göre 3, 5 veya 7 tur süren bir **for** döngüsü çalıştırır. Her turda **i % 2 == 0** kontrolüyle çift numaralı turlarda çarpan 2 yapılır ve hasar iki katına çıkar. Korunma eşiği oyuncunun durumuna bağlıdır: modül hasarı 50'nin altında ve sağlık 50'nin üzerindeyse eşik 40, değilse 20'dir. Zar, IF–ELSE IF zinciriyle dört olaydan birine eşlenir: korunma, can kaybı, oksijen sızıntısı veya modül hasarı.

```
for (i = 1; i <= firtinaTurSayisi; i++) {
    if (i % 2 == 0) carpan = 2; else carpan = 1;
    kayip = firtinaHasari * carpan;
    zar = rand() % 100;
    if (zar < korunmaEsigi) {
        /* zararsiz korunma */
    } else if (zar < korunmaEsigi + 20) {
        saglik -= kayip;  toplamSaglikKaybi += kayip;  hasarliTur++;
    } else if (zar < korunmaEsigi + 45) {
        if (kilitMuhurlu == 1) {
            continue;              /* sizinti bu tur atlanir */
        }
        oksijen -= kayip;  toplamOksijenKaybi += kayip;  hasarliTur++;
    } else {
        modulHasari += kayip;  hasarliTur++;
    }
    if (saglik == 0 || oksijen == 0) {
        break;                     /* firtina hemen sonlanir */
    }
}
```

Hava kilidi daha önce mühürlendiyse oksijen sızıntısı olayı **continue** ile atlanır ve döngü bir sonraki tura geçer. Her turdan sonra değerler 0'ın altına düşmeyecek şekilde sınırlandırılır; sağlık veya oksijen 0 olursa **break** ile döngü derhal sonlandırılır. Döngüden sonra hasar alınan tur sayısı, toplam sağlık kaybı ve toplam oksijen kaybı özet olarak yazdırılır.

7. Hava Kilidi Şifresi: DO–WHILE Döngüsü

Şifre sabit değildir, o günün sayısından aritmetik olarak türetilir: **sifre = 'A' + (gun % 26)**. Örneğin 1. günde şifre 'B', 2. günde 'C' olur. Oyuncuya yalnızca ipucu gösterilir. Küçük harf girişi **tahmin - 'a' + 'A'** işlemiyle büyük harfe çevrilir. A–Z dışındaki bir karakter girilirse uyarı verilir ve **continue** ile deneme hakkı düşmeden döngünün koşuluna gidilir.

```
do {
    scanf(" %c", &tahmin);
    if (tahmin >= 'a' && tahmin <= 'z') tahmin = tahmin - 'a' + 'A';
```

```
    if (tahmin < 'A' || tahmin > 'Z') continue;   /* deneme sayilmaz */
    deneme++;
    if (tahmin != sifre) {
        enerji -= SIFRE_ENERJI;                    /* -5 enerji */
        if (tahmin < sifre) printf("ILERI git");
        else                printf("GERI git");
    }
} while (tahmin != sifre && deneme < MAKS_DENEME);

if (tahmin == sifre) kilitMuhurlu = 1;
else                 modulHasari += 20;
```

Döngü koşulu, şifrenin yanlış olmasını ve deneme hakkının bitmemiş olmasını tek bir **&&** ifadesinde kontrol eder. Harfler ASCII değerleriyle karşılaştırıldığı için **tahmin < sifre** ifadesi, girilen harfin alfabede şifreden önce geldiğini gösterir.

8. Sınırlandırma, Zaman Akışı ve Oyun Sonu

switch bloğundan sonra, hangi komut çalışırsa çalışsın sağlık, enerji, oksijen ve modül hasarı IF yapılarıyla 0–100 aralığına çekilir. Komut tur harcadıysa tur sayacı artırılır ve **tur % 3 == 0** olduğunda yeni gün başlar. Gün başında oksijen 8 azalır; modül hasarı 70'in üzerindeyse ek olarak 5 sağlık kaybedilir.

```
if (turGecti == 1) {
    tur++;
    if (tur % 3 == 0) {
        gun++;
        if (gun > SON_GUN) {
            oyunBitti = 1;                 /* 15. gun tamamlandi */
        } else {
            oksijen -= 8;
            if (modulHasari > 70) saglik -= 5;
        }
    }
    if (saglik <= 0 || oksijen <= 0) {
        oyunBitti = 2;                     /* kaybedildi */
    }
}
```

Oyun sonu kontrolü her turun sonunda yapılır. Gün sayacı 16'ya ulaştığında 15. gün tamamlanmış sayılır ve kurtarma ekibi gelir. Oyun hangi şekilde biterse bitsin, döngüden çıkıldıktan sonra sonuç raporu yazdırılır: hayatta kalınan gün, toplam tur, son durum değerleri ve puan.

9. Puan Hesabı

```
hayattaKalinanGun = gun - 1;     /* tamamlanan gun sayisi */
puan = (hayattaKalinanGun * 100) + saglik + oksijen - modulHasari;
```

Hayatta kalınan gün olarak, devam eden gün değil tamamlanan gün sayısı kullanılır. Gün katsayısı 100 seçilmiştir; çünkü sağlık ve oksijen en fazla 100 puan katkı sağlayabilir ve hayatta kalınan her günün bu değerlerden daha önemli olması amaçlanmıştır. Modül hasarı, istasyonun bırakıldığı durumu gösterdiği için puandan çıkarılır. Örnek: 15 gün, sağlık 70, oksijen 55, hasar 30 ise puan = 1500 + 70 + 55 − 30 = 1595.

10. Örnek Çalışma

```
Zorluk seviyesi secin (K: Kolay, N: Normal, Z: Zor): x
[UYARI] Gecersiz zorluk seviyesi: 'x'. Lutfen K, N veya Z girin.
Zorluk seviyesi secin (K: Kolay, N: Normal, Z: Zor): n
--- Gun 1 | Tur 0 | Saglik 100 | Enerji 90 | Oksijen 80 ---
```

```
Komut girin (G/O/T/D/U/K/H/Q): w
[UYARI] Tanimsiz komut: 'w'. Tur harcanmadi.
Komut girin (G/O/T/D/U/K/H/Q): g
Durumun iyi, genis bir alani tarayabilirsin.
Zar: 60 / Parca esigi: 70 -> BASARILI (yedek parca bulundu)
Yaralanma zari: 70 / Esik: 10 (hasar: 20) -> Saglam dondun.
Komut girin (G/O/T/D/U/K/H/Q): o
Kac birim su kullanilsin? (2 - 14): 3
3 tek sayi: 1 birim su artik olarak depoda kaldi.
2 su -> 3 oksijen uretildi (2 su = 3 oksijen). Kalan su: 12
```

11. Derleme ve Çalıştırma

```
gcc mars_kolonisi.c -o mars_kolonisi
./mars_kolonisi
```

Program **gcc -Wall -Wextra -std=c99 -pedantic** seçenekleriyle uyarısız olarak derlenmektedir.

GitHub Deposu: https://github.com/MavlyudaAlmazova/mars-kolonisi-simulatoru